

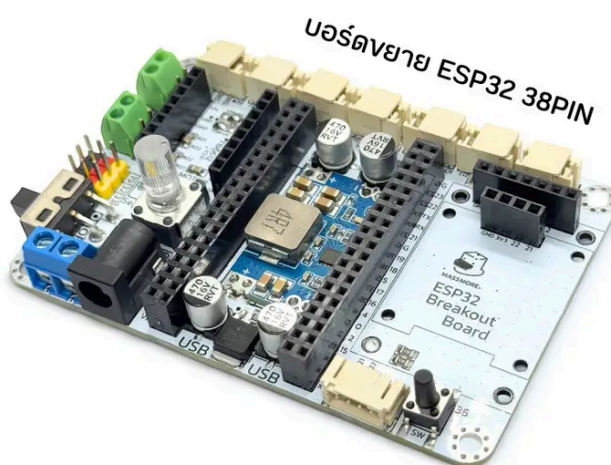
ชุดฝึกหุ่นยนต์ ESP32

ESP32 38PIN Breakout Board — บอร์ดทดลอง Prototype

เอกสารประกอบการเรียนรู้ฉบับปฏิบัติทีละขั้นตอน | SKU-1015 โดย Massmore

ชุดฝึกหุ่นยนต์ ESP32

บอร์ดทดลอง Prototype



อุปกรณ์ในชุดเรียนรู้	SKU
1. บอร์ดขยาย ESP32 38PIN Breakout Board	SKU1015
2. บอร์ดพัฒนา ESP32-WROOM-32E 38PIN (CH340)	SKU1016
3. โมดูลขับมอเตอร์ 2CH TB67H450	SKU1002
4. จอแสดงผล TFT LCD IPS 240x240 ST7789	SKU0014

เอกสารเวอร์ชัน 3.0 | กรกฎาคม 2569 | www.massmore.shop

สารบัญ (Contents)

หน้า

ส่วนที่ 1 — แนะนำชุดฝึกและอุปกรณ์ทั้ง 4 รายการ	3
1.1 รายการที่ 1: บอร์ดขยาย ESP32 38PIN Breakout Board (SKU1015)	3
1.2 รายการที่ 2: บอร์ดพัฒนา ESP32-WROOM-32E 38PIN (SKU1016)	6
1.3 รายการที่ 3: โมดูลขับเคลื่อนมอเตอร์ 2CH TB67H450 (SKU1002)	8
1.4 รายการที่ 4: จอแสดงผล TFT LCD IPS 240×240 ST7789 (SKU0014)	10
1.5 สรุปการใช้ขาสัญญาณของอุปกรณ์แต่ละรายการ	11
ส่วนที่ 2 — การติดตั้งโปรแกรมและเครื่องมือ 6 ขั้นตอน	13
ขั้นตอนที่ 1 Arduino IDE · ขั้นตอนที่ 2 Driver CH340	13
ขั้นตอนที่ 3 ESP32 Package	14
ขั้นตอนที่ 4 ตั้งค่าบอร์ด · ขั้นตอนที่ 5 อัปโหลดโปรแกรมแรก	15
ขั้นตอนที่ 6 ไลบรารี	16
ส่วนที่ 3 — กิจกรรมพื้นฐาน 10 กิจกรรม	17
กิจกรรมที่ 1 การสื่อสารผ่านพอร์ตอนุกรม · กิจกรรมที่ 2 การควบคุม LED	17
กิจกรรมที่ 3 การอ่านค่าสวิตช์ · กิจกรรมที่ 4 การอ่านค่าอนาล็อกจาก VR	18
กิจกรรมที่ 5 การอ่านอินพุต 7 ช่อง · กิจกรรมที่ 6 การสร้างสัญญาณ PWM	19
กิจกรรมที่ 7 การควบคุมเซอร์โวมอเตอร์ · กิจกรรมที่ 8 การควบคุมมอเตอร์กระแสตรง	20
กิจกรรมที่ 9 การสแกนอุปกรณ์บนบัส I2C	21
กิจกรรมที่ 10 การเชื่อมต่อ WiFi	22
ส่วนที่ 4 — กิจกรรมขั้นสูงและโปรเจกต์ประยุกต์	23
กิจกรรมที่ 11 จอ TFT · กิจกรรมที่ 12 จอ OLED · กิจกรรมที่ 13 VR ควบคุมเซอร์โว	23
กิจกรรมที่ 14 โปรแกรมทดสอบ Factory Test · โปรเจกต์ประยุกต์: รถหุ่นยนต์ 2 ล้อ	24
ส่วนที่ 5 — แหล่งข้อมูลอ้างอิงและการแก้ไขปัญหา	26
5.1 ลิงก์สินค้า · 5.2 GitHub และไลบรารี · 5.3 เอกสาร Datasheet	26
5.4 การแก้ไขปัญหาที่พบบ่อย · 5.5 ข้อเสนอแนะเพิ่มเติม	27

ส่วนที่ 1 — แนะนำชุดฝึกและอุปกรณ์ทั้ง 4 รายการ

ชุดฝึกหุ่นยนต์ ESP32 ประกอบด้วยอุปกรณ์ 4 รายการซึ่งออกแบบให้ทำงานร่วมกันเป็นระบบเดียว โดยอาจเปรียบเทียบหน้าที่ของอุปกรณ์แต่ละรายการกับส่วนประกอบของหุ่นยนต์ได้ดังตารางต่อไปนี้

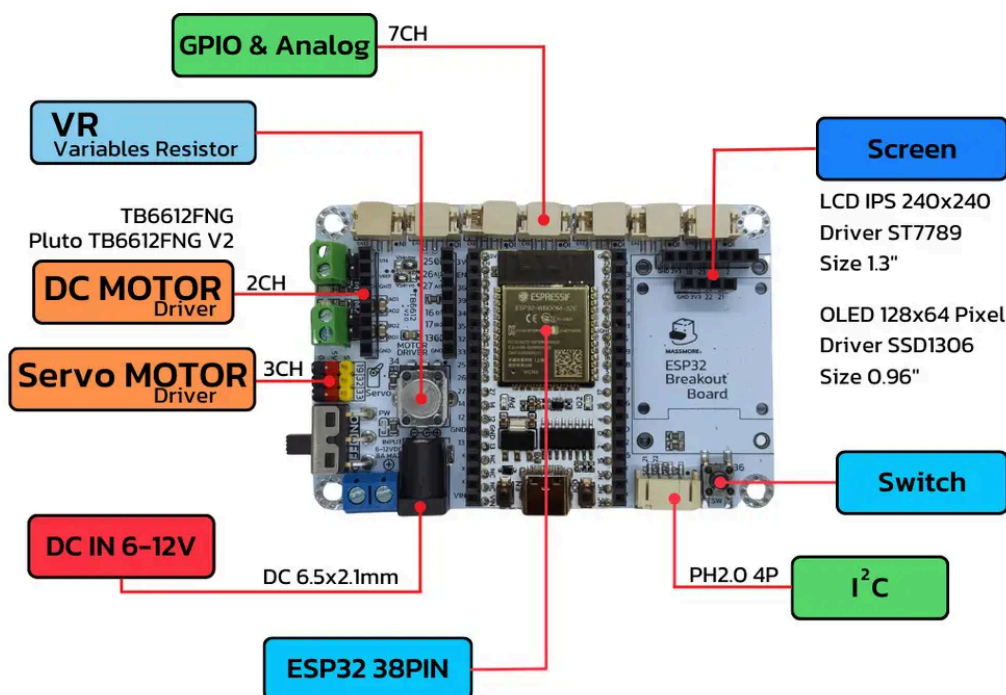
อุปกรณ์	เปรียบเทียบ	หน้าที่ในระบบ
ESP32-WROOM-32E (SKU1016)	หน่วยประมวลผล (สมอง)	ประมวลผล ตัดสินใจ รับ-ส่งข้อมูลผ่าน WiFi/Bluetooth และสั่งการอุปกรณ์ทั้งหมด
Breakout Board (SKU1015)	โครงสร้างและระบบเชื่อมต่อ	เป็นจุดเชื่อมต่ออุปกรณ์ทุกชิ้นเข้าด้วยกัน พร้อมจ่ายไฟเลี้ยงให้ทั้งระบบ
TB67H450 Motor Driver (SKU1002)	ระบบขับเคลื่อน (กล้ามเนื้อ)	รับคำสั่งจากหน่วยประมวลผลไปขับมอเตอร์ตามทิศทางและความเร็วที่กำหนด
จอ TFT ST7789 (SKU0014)	ส่วนแสดงผล	แสดงข้อมูล ตัวเลข กราฟ และสถานะการทำงานของระบบ

 **ความรู้เพิ่มเติม:** ลำดับการไหลของข้อมูลในระบบ: เซนเซอร์ / VR / สวิตช์ ส่งสัญญาณเข้า ESP32 เพื่อประมวลผล จากนั้น ESP32 สั่งการ Motor Driver ให้ขับมอเตอร์ และสั่งจอแสดงผลให้แสดงข้อมูลทั้งหมดเชื่อมต่อกันผ่านลายวงจรบน Breakout Board โดยไม่ต้องเดินสายไฟเพิ่มเติม

1.1 รายการที่ 1: บอร์ดขยาย ESP32 38PIN Breakout Board (SKU1015)

บอร์ดหลักของชุดฝึก ราคา 590 บาท — บอร์ดขยายสำหรับ ESP32 ออกแบบมาเพื่ออำนวยความสะดวกในการพัฒนาโปรเจกต์หุ่นยนต์และระบบควบคุมต่าง ๆ รองรับการเชื่อมต่อมอเตอร์ เซอร์โวมอเตอร์ จอแสดงผล และโมดูลเสริมอื่น ๆ เหมาะสำหรับการสร้างและทดสอบโปรเจกต์ IoT หุ่นยนต์ และระบบอัตโนมัติ

ชุดฝึกหุ่นยนต์ ESP32



ภาพที่ 1: ส่วนประกอบบนบอร์ด SKU1015

หัวข้อ	รายละเอียด
ซีอิกเก็ต MCU	รองรับโมดูล ESP32 38PIN (NodeMCU-32S / DevKitC)
Motor Driver	ซีอิกเก็ต 2 ช่อง สำหรับโมดูล TB6612FNG / TB67H450
Servo	3 ช่อง (GPIO19, 32, 33) ไฟเลี้ยง 5V จากบอร์ด
จอแสดงผล	ซีอิกเก็ต TFT ST7789 (SPI) และ OLED SSD1306 (I2C)
GPIO & Analog	7 ช่อง (IO1–IO7) คอนเนกเตอร์แบบ S-V-G พร้อม VR และสวิตช์
I2C	พอร์ต PH2.0 4P (SDA=GPIO21, SCL=GPIO22)
ไฟเลี้ยงเข้า	DC IN 6–12V (แฉัก 6.5×2.1 มม.) สูงสุด 8A พร้อมสวิตช์เปิด-ปิด
DC-DC Step Down	เอาต์พุต 5V / 3A รองรับโหลดกำลังสูง
ขนาดบอร์ด	90 × 60 มิลลิเมตร

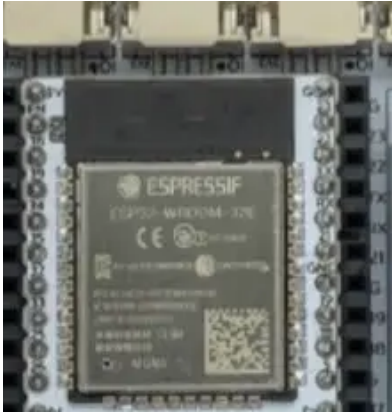
ตารางขาสัญญาณ (Pinout) — ใช้อ้างอิงในทุกกิจกรรมของเอกสารฉบับนี้

กลุ่ม	ชื่อขา	GPIO	หมายเหตุ
DC Motor	M1_IN1 / M1_IN2	27 / 26	มอเตอร์ช่องที่ 1
DC Motor	M2_IN1 / M2_IN2	16 / 17	มอเตอร์ช่องที่ 2
Servo	Servo1 / 2 / 3	19 / 32 / 33	เซอร์โว 3 ช่อง
Analog In	IO1–IO7	25, 13, 12, 14, 15, 5, 35	IO6 (GPIO5) ใช้งานแบบดิจิทัล, IO7 (GPIO35) รับสัญญาณอย่างเดียว
I2C	SDA / SCL	21 / 22	จอ OLED และเซนเซอร์
จอ SPI	CLK / MOSI / RES / DC	18 / 23 / 4 / 2	จอ TFT ST7789
VR	VR	34	ตัวต้านทานปรับค่าได้ (รับสัญญาณอย่างเดียว)
Switch	SW	36	สวิตช์กด Active LOW (รับสัญญาณอย่างเดียว)

⚠️ ข้อควรระวัง: GPIO34, 35 และ 36 เป็นขารับสัญญาณอย่างเดียว (Input-only) ไม่สามารถใช้เป็นเอาต์พุตได้ และ GPIO2 ใช้ร่วมกับขา DC ของจอ TFT — หากติดตั้งจอไว้ ไม่ควรใช้ GPIO2 ทำงานอื่นพร้อมกัน

1.2 รายการที่ 2: บอร์ดพัฒนา ESP32-WROOM-32E 38PIN (SKU1016)

หน่วยประมวลผลหลักของชุดฝึก ราคา 350 บาท — บอร์ดพัฒนา ESP32 DevKitC แบบ 38 ขา ใช้โมดูล ESP32-WROOM-32E จาก Espressif Systems ภายในเป็นชิป ESP32-D0WD-V3 (Revision 3) ซึ่งได้รับการปรับปรุงข้อบกพร่องจากรุ่นก่อนหน้า ทำให้ระบบมีความเสถียรและความร้อนสะสมต่ำลง มาพร้อมชิปแปลงสัญญาณ USB-to-Serial เบอร์ CH340 และพอร์ต USB Type-C



รายการ	ข้อมูลจำเพาะ
CPU	Dual-Core Xtensa LX6 @ 240 MHz
หน่วยความจำ	SRAM 520 KB + Flash 4 MB
WiFi	802.11 b/g/n (2.4 GHz)
Bluetooth	v4.2 BR/EDR + BLE
USB to Serial	CH340 (USB Type-C)
ไฟเลี้ยง	5–12V (ผ่าน USB หรือขา VIN)
ระดับลอจิก I/O	3.3V (ห้ามป้อนสัญญาณ 5V เข้าขาโดยตรง)
กระแสใช้งาน	ประมาณ 80 mA (สูงสุด 500 mA ขณะส่ง WiFi)

รุ่นที่มีจำหน่าย

- ESP32-WROOM-32E — สายอากาศแบบลายวงจรบนบอร์ด (PCB Antenna) เหมาะกับการใช้งานทั่วไป (รุ่นที่ใช้ในชุดฝึกนี้)
- ESP32-WROOM-32UE — มีจุดต่อสายอากาศภายนอกแบบ IPEX/U.FL เหมาะกับการติดตั้งในตู้ควบคุมหรืองานที่ต้องการระยะสัญญาณไกล (ต้องต่อสายอากาศก่อนเปิดใช้ WiFi ทุกครั้ง)

วิธีติดตั้งเข้ากับ Breakout Board

1. สังเกตซ็อกเก็ตสีดำ 2 แถวบริเวณกลางบอร์ดขยาย และหมายเลข GPIO ที่พิมพ์กำกับไว้ข้างซ็อกเก็ต
2. หันพอร์ต USB Type-C ของบอร์ด ESP32 ไปทางด้านล่างของบอร์ดขยาย (ด้านที่พิมพ์ข้อความ USB)
3. กดโมดูลลงช้า ๆ ให้ขาทั้ง 38 ขาตรงร่องครบทุกขา แล้วกดยืนยันเบา ๆ ทั้งสองด้าน
4. เชื่อมต่อสาย USB Type-C เพื่ออัปโหลดโปรแกรม — ไฟแสดงสถานะบนโมดูลจะติดเมื่อมีไฟเลี้ยง

โปรแกรมตัวอย่าง: ตรวจสอบข้อมูลชิป ESP32

```
// ตรวจสอบข้อมูลชิป ESP32 – อัปโหลดแล้วเปิด Serial Monitor (115200)
#include <WiFi.h>

void setup() {
  Serial.begin(115200);
  delay(1000);
  Serial.printf("Chip Model : %s\n", ESP.getChipModel()); // ESP32-D0WD-V3
  Serial.printf("Cores      : %d\n", ESP.getChipCores()); // 2 คอร์
  Serial.printf("CPU Freq   : %d MHz\n", getCpuFrequencyMhz());
  Serial.printf("Flash      : %d MB\n", ESP.getFlashChipSize()/1024/1024);
  Serial.printf("MAC WiFi    : %s\n", WiFi.macAddress().c_str());
}
void loop() {}
```

 **ความรู้เพิ่มเติม:** ESP32 มีหน่วยประมวลผล 2 คอร์ โดยคอร์ 0 ทำหน้าที่ดูแลงาน WiFi/Bluetooth และคอร์ 1 ประมวลผลโปรแกรม Arduino จึงสามารถใช้งานเครือข่ายไร้สายพร้อมกับควบคุมหุ่นยนต์ได้อย่างต่อเนื่อง

1.3 รายการที่ 3: โมดูลขับเคลื่อนมอเตอร์ 2CH TB67H450 (SKU1002)

ระบบขับเคลื่อนของชุดฝึก ราคา 159 บาท — โมดูลขับเคลื่อนมอเตอร์กระแสตรงประสิทธิภาพสูง ใช้ชิป TB67H450 ของ TOSHIBA ซึ่งมีสมรรถนะสูงกว่าโมดูล TB6612 รุ่นเดิมอย่างมาก ควบคุมมอเตอร์กระแสตรงได้พร้อมกัน 2 ช่อง หรือสเต็ปเปอร์มอเตอร์ 1 ตัว เหมาะสำหรับหุ่นยนต์ รถบังคับ และเครื่องจักรขนาดเล็ก

 TB6612FNG iTo TB6612FNG V2	รายการ	ข้อมูลจำเพาะ
	ชิปหลัก	TOSHIBA TB67H450 (Dual H-Bridge)
	แรงดันมอเตอร์ (VM)	4.5V – 28V
	แรงดันลอจิก (VCC)	2.7V – 5.5V (รองรับ 3.3V ของ ESP32)
	กระแสต่อเนื่อง	3.5A ต่อช่อง (สูงสุด 4.5A)
	ความถี่ PWM สูงสุด	100 kHz
	โหมดการทำงาน	Forward / Reverse / Brake / Standby
	ระบบป้องกัน	TSD (อุณหภูมิเกิน), ISD (กระแสเกิน), UVLO (แรงดันตก)

การเปรียบเทียบกับ TB6612FNG รุ่นเดิม

คุณสมบัติ	TB67H450 (รุ่นนี้)	TB6612FNG (รุ่นเดิม)
กระแสต่อเนื่อง	สูงสุด 3.5A	1.2A
แรงดันมอเตอร์	4.5 – 28V	4.5 – 13.5V
ความร้อนสะสม	ต่ำกว่า (RDS(on) 0.5Ω)	สูงกว่า

วิธีติดตั้งเข้ากับ Breakout Board

1. บัดกรี Pin Header ที่ให้มากับโมดูล (กรณียังไม่ได้บัดกรี) หรือเลือกซื้อรุ่นที่บัดกรีสำเร็จแล้ว
2. เสียบโมดูลลงซ็อกเก็ตที่พิมพ์ข้อความ MOTOR DRIVER บนบอร์ดขยาย โดยให้ทิศทางตรงตามชิลด์สกรีน
3. ต่อสายมอเตอร์ตัวที่ 1 เข้าเทอร์มินอล M1 และมอเตอร์ตัวที่ 2 เข้าเทอร์มินอล M2
4. ต่อไฟเลี้ยง DC IN 6–12V เข้าบอร์ด แล้วเปิดสวิตช์ ON — บอร์ดจะจ่ายไฟให้โมดูลโดยอัตโนมัติ

โปรแกรมตัวอย่าง: ควบคุมมอเตอร์เดินหน้า-ถอยหลัง

```
// TB67H450: ควบคุมด้วยขา IN1/IN2 ต่อช่อง (จ่ายสัญญาณ PWM เพื่อปรับความเร็ว)
#define M1_IN1 27
#define M1_IN2 26

void setup() {
  ledcSetup(0, 1000, 8); ledcAttachPin(M1_IN1, 0); // PWM 1kHz 8-bit
  ledcSetup(1, 1000, 8); ledcAttachPin(M1_IN2, 1);
}

void loop() {
  ledcWrite(0, 200); ledcWrite(1, 0); delay(2000); // เดินหน้า (ความเร็ว 200/255)
  ledcWrite(0, 0); ledcWrite(1, 0); delay(1000); // หยุด (Standby)
  ledcWrite(0, 0); ledcWrite(1, 200); delay(2000); // ถอยหลัง
  ledcWrite(0, 255); ledcWrite(1, 255); delay(500); // เบรก (Brake)
}
```

 **ข้อควรระวัง:** มอเตอร์ใช้กระแสไฟฟ้าสูง ห้ามใช้ไฟเลี้ยงจากพอร์ต USB เพียงอย่างเดียว — ต้องต่ออะแดปเตอร์หรือแบตเตอรี่ 6–12V เข้าช่อง DC IN ทุกครั้ง มิฉะนั้นพอร์ต USB ของคอมพิวเตอร์อาจเสียหาย

1.4 รายการที่ 4: จอแสดงผล TFT LCD IPS 240×240 ST7789 (SKU0014)

ส่วนแสดงผลของชุดฝึก ราคา 160 บาท — จอแสดงผลสีชนิด TFT LCD เทคโนโลยี IPS ให้มุมมองกว้างถึง 178 องศา สีสั่นคมชัดไม่ผิดเพี้ยนแม้มองจากด้านข้าง ความละเอียด 240×240 พิกเซล แสดงผลได้ 65,000 สี (RGB 65K) สื่อสารผ่านโปรโตคอล SPI ซึ่งใช้สายสัญญาณจำนวนน้อย ช่วยประหยัดขา GPIO

 LCD IPS 240×240	รายการ	ข้อมูลจำเพาะ
	ชิปขับจอ	ST7789
	ความละเอียด	240 × 240 พิกเซล (RGB 65K สี)
	เทคโนโลยี	IPS มุมมองกว้าง 178°
	อินเทอร์เฟซ	SPI 4-wire
	ไฟเลี้ยง	3.3V DC
	รุ่น 1.3 นิ้ว	7 ขา: GND VCC SCL SDA RES DC BLK (28×39 มม.)
	รุ่น 1.54 นิ้ว	8 ขา: เพิ่มขา CS (32×43.6 มม.)

หน้าที่ของขาสัญญาณแต่ละขา

ขา	หน้าที่	ต่อกับขาบนบอร์ด
GND	กราวด์	GND
VCC	ไฟเลี้ยง 3.3V	3V3
SCL	สัญญาณนาฬิกา SPI (Clock)	GPIO18
SDA	สัญญาณข้อมูล SPI (MOSI)	GPIO23
RES	รีเซ็ตจอ (Active Low)	GPIO4
DC	เลือกโหมดส่งข้อมูลหรือคำสั่ง	GPIO2
BLK/BL	ควบคุมไฟส่องหลัง (ต่อ 3.3V เพื่อเปิด)	3V3

วิธีติดตั้งเข้ากับ Breakout Board

- เสียบจอลงซ็อกเก็ตจอด้านขวาบนของบอร์ด โดยเทียบชื่อขาบนจอให้ตรงกับชิลด์สกรีน (GND ตรงกับ GND) — บอร์ดออกแบบให้เสียบได้พอดีโดยไม่ต้องเดินสายเพิ่มเติม

- ติดตั้งไลบรารี TFT_eSPI จากโฟลเดอร์ Arduino_Library ของโปรเจกต์ (มีการตั้งค่า User_Setup.h ไว้เรียบร้อยแล้ว)
- อัปโหลดโปรแกรมทดสอบด้านล่าง — จอจะแสดงข้อความและรูปวงกลมสี

โปรแกรมตัวอย่าง: การใช้งานจอครั้งแรก

```
#include <TFT_eSPI.h>
TFT_eSPI tft = TFT_eSPI(); // ใ้ค่าจาก User_Setup.h ที่ตั้งไว้แล้ว

void setup() {
  tft.init();
  tft.setRotation(0);
  tft.fillScreen(TFT_BLACK);          // พื้นหลังสีดำ
  tft.setTextColor(TFT_GREEN, TFT_BLACK);
  tft.setTextSize(2);
  tft.setCursor(30, 30);
  tft.println("Hello Massmore!");
  tft.fillCircle(120, 140, 50, TFT_RED); // วงกลมสีแดงกลางจอ
  tft.drawCircle(120, 140, 60, TFT_CYAN); // เส้นขอบวงกลมสีฟ้า
}
void loop() {}
```

อุปกรณ์ทางเลือก: บอร์ดมีซ็อกเก็ตสำหรับจอ OLED 128×64 SSD1306 (I2C, Address 0x3C) ซึ่งเป็นจอราคาประหยัด ใช้ไลบรารี Adafruit_SSD1306 เหมาะสำหรับการแสดงข้อความและตัวเลข



ภาพที่ 2: จอ OLED 128×64 SSD1306 (อุปกรณ์เสริม)

1.5 สรุปการใช้ขาสัญญาณของอุปกรณ์แต่ละรายการ

อุปกรณ์	ขา GPIO ที่ใช้	ข้อขัดแย้งของขาสัญญาณ
TB67H450 (มอเตอร์ 2 ช่อง)	27, 26, 16, 17	ไม่มี — ใช้ร่วมกับอุปกรณ์อื่นได้ทั้งหมด
เซอร์โว 3 ช่อง	19, 32, 33	ไม่มี
จอ TFT ST7789	18, 23, 4, 2	GPIO2 ใช้ร่วมกับ LED บนโมดูล ESP32 (ไม่กระทบการใช้งาน)
จอ OLED / I2C	21, 22	ไม่มี

อุปกรณ์	ขา GPIO ที่ใช้	ข้อขัดแย้งของขาสัญญาณ
เซนเซอร์ IO1–IO7	25, 13, 12, 14, 15, 5, 35	ไม่มี
VR และสวิตช์	34, 36	ไม่มี

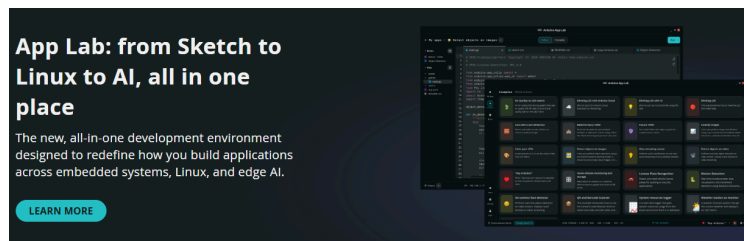
✅ **ผลการทำงานที่ถูกต้อง:** บอร์ดออกแบบให้อุปกรณ์ทุกรายการใช้ขาสัญญาณแยกจากกัน จึงสามารถติดตั้งมอเตอร์ เซอร์โว จอแสดงผล และเซนเซอร์พร้อมกันทั้งหมดได้โดยไม่เกิดข้อขัดแย้งของขาสัญญาณ

ส่วนที่ 2 — การติดตั้งโปรแกรมและเครื่องมือ 6 ขั้นตอน

ภาพรวมขั้นตอน: ขั้นตอนที่ 1 ติดตั้ง Arduino IDE → ขั้นตอนที่ 2 ติดตั้ง Driver CH340 → ขั้นตอนที่ 3 เพิ่ม ESP32 Package → ขั้นตอนที่ 4 ตั้งค่าบอร์ดและพอร์ต → ขั้นตอนที่ 5 อัปโหลดโปรแกรมแรก → ขั้นตอนที่ 6 ติดตั้งไลบรารี

ขั้นตอนที่ 1 — ติดตั้ง Arduino IDE 2

1. เข้าเว็บไซต์ <https://www.arduino.cc/en/software> และดาวน์โหลดตามระบบปฏิบัติการ: Windows (.exe) / macOS (.dmg) / Linux (.ApImage)
2. ติดตั้งตามขั้นตอนปกติ (Next → Install → Finish) จากนั้นเปิดโปรแกรม Arduino IDE 2



Bring Your Projects to Life with Arduino Software

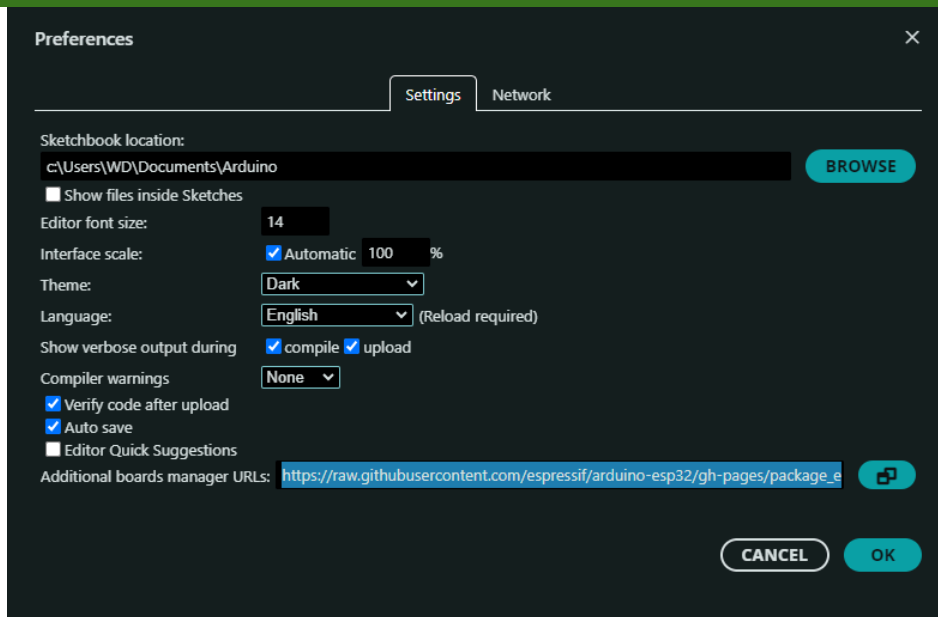


ขั้นตอนที่ 2 — ติดตั้ง Driver CH340

1. Windows: ดาวน์โหลดไฟล์ CH341SER.EXE จาก https://www.wch-ic.com/downloads/CH341SER_EXE.html แล้วกดปุ่ม INSTALL
2. เชื่อมต่อสาย USB-C จากบอร์ดเข้าคอมพิวเตอร์ → เปิด Device Manager → ตรวจสอบว่าปรากฏ "USB-SERIAL CH340 (COMx)" ในหมวด Ports
3. macOS: ติดตั้งไฟล์ .pkg จาก [wch-ic.com](https://www.wch-ic.com) แล้วกดอนุญาตใน Privacy & Security | Linux: มี Driver ในระบบแล้ว เพิ่มสิทธิ์ด้วยคำสั่ง `sudo usermod -aG dialout $USER`

✓ **วิธีตรวจสอบพอร์ต:** ถอดสาย USB → บันทึกรายชื่อพอร์ตที่มีอยู่ → เชื่อมต่อสายอีกครั้ง → พอร์ตใหม่ที่ปรากฏขึ้นคือพอร์ตของบอร์ด

ขั้นตอนที่ 3 — เพิ่ม ESP32 Board Package



1. เปิดเมนู File → Preferences แล้ววาง URL ต่อไปนี้ในช่อง Additional boards manager URLs:

https://raw.githubusercontent.com/espressif/arduino-esp32/gh-pages/package_esp32_index.json

2. ไปที่ Tools → Board → Boards Manager ค้นหาคำว่า "esp32" และเลือก esp32 by Espressif Systems

3. เลือกเวอร์ชัน 2.0.17 แล้วกด Install (ขนาดประมาณ 300–500 MB ใช้เวลา 3–10 นาที)

⚠ **ข้อควรระวัง:** จำเป็นต้องใช้เวอร์ชัน 2.0.x เท่านั้น เนื่องจากโปรแกรมตัวอย่างในเอกสารฉบับนี้ใช้คำสั่ง ledcSetup/ledcAttachPin ของ core 2.x หากติดตั้งเวอร์ชัน 3.x โปรแกรมส่วน PWM จะไม่สามารถคอมไพล์ได้

ขั้นตอนที่ 4 — ตั้งค่าบอร์ดและเลือกพอร์ต

1. ติดตั้งโมดูล ESP32 ลงบนบอร์ดขยาย (พอร์ต USB ด้านล่าง) แล้วเชื่อมต่อสาย USB
2. เลือกเมนู Tools → Board → esp32 → ESP32 Dev Module
3. ตั้งค่า: Upload Speed 921600, CPU 240MHz, Flash 4MB, Partition Default, PSRAM Disabled
4. เลือก Tools → Port ให้ตรงกับพอร์ตของบอร์ด (Windows: COMx | Linux: /dev/ttyUSB0)



ขั้นตอนที่ 5 — อัปโหลดโปรแกรมแรก

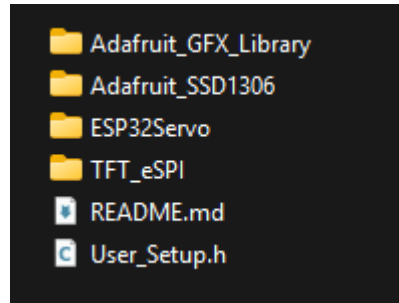
เปิดไฟล์ Arduino_Example_Basic/01_HelloWorld_Serial กดปุ่ม Verify เพื่อคอมไพล์ รอจนแสดง Done compiling จากนั้นกดปุ่ม Upload รอจนแสดง Done uploading แล้วเปิด Serial Monitor (Ctrl+Shift+M) ตั้งค่า 115200 baud

✅ **ผลการทำงานที่ถูกต้อง:** Serial Monitor แสดงข้อความ "SKU-1015 ESP32 Breakout Board — Hello World!" พร้อมข้อมูลชิป และค่า Uptime เพิ่มขึ้นทุกวินาที แสดงว่าระบบพร้อมใช้งาน

💡 **ข้อแนะนำ:** กรณีอัปโหลดค้างที่ข้อความ Connecting... ให้กดปุ่ม BOOT (IO0) บนโมดูลค้างไว้จนเริ่มอัปโหลดแล้วจึงปล่อย และตรวจสอบว่าสาย USB เป็นสายรับ-ส่งข้อมูล มิใช่สายสำหรับชาร์จเพียงอย่างเดียว

ขั้นตอนที่ 6 — ติดตั้งไลบรารี

คัดลอกโฟลเดอร์ทั้งหมดจาก Arduino_Library/ ของโปรเจกต์ ไปวางที่ Documents/Arduino/libraries/ แล้วเปิดโปรแกรม Arduino IDE ใหม่ ไลบรารีประกอบด้วย TFT_eSPI (ตั้งค่าจอไว้แล้ว), Adafruit_GFX, Adafruit_SSD1306 และ ESP32Servo



⚠️ ข้อควรระวัง: หากติดตั้ง TFT_eSPI จาก Library Manager ด้วยตนเอง จะต้องแก้ไขไฟล์ User_Setup.h ใหม่ดังนี้: ST7789_DRIVER, 240×240, MOSI=23, SCLK=18, DC=2, RST=4, CS=-1, TFT_RGB_ORDER=BGR, TFT_INVERSION_ON

ส่วนที่ 3 — กิจกรรมพื้นฐาน 10 กิจกรรม

โปรดปฏิบัติตามลำดับกิจกรรมที่ 1 ถึง 10 ทุกกิจกรรมมีโปรแกรมพร้อมใช้งานในโฟลเดอร์ Arduino_Example_Basic/ โดยมีขั้นตอนหลักคือ เปิดไฟล์ อัปโหลดโปรแกรม และสังเกตผลการทำงาน

กิจกรรมที่ 1 — การสื่อสารผ่านพอร์ตอนุกรม (Hello World)

 **วัตถุประสงค์:** เรียนรู้การอัปโหลดโปรแกรมและการอ่านข้อความจากบอร์ดผ่าน Serial Monitor

 **อุปกรณ์ที่ใช้:** บอร์ดชุดฝึกและสาย USB-C (ไม่ต้องใช้อุปกรณ์เสริม)

1. เปิดไฟล์ 01_HelloWorld_Serial.ino แล้วกด Upload
2. เปิด Serial Monitor ตั้งค่า 115200 baud แล้วกดปุ่ม EN/RST บนโมดูล
3. สังเกตข้อมูลชิปและค่า Uptime ที่เพิ่มขึ้นทีละ 1 ทุกวินาที

 **ผลการทำงานที่ถูกต้อง:** ปรากฏชื่อบอร์ด ข้อมูล CPU/Flash และบรรทัด [1] Uptime: 1 s ซึ่งเพิ่มขึ้นอย่างต่อเนื่อง

 **ความรู้เพิ่มเติม:** คำสั่ง Serial.begin(115200) เป็นการเปิดช่องทางสื่อสารระหว่างบอร์ดกับคอมพิวเตอร์ โดยค่า 115200 คืออัตราเร็วในการรับ-ส่งข้อมูล (baud rate) ซึ่งต้องตั้งให้ตรงกันทั้งสองฝั่ง

กิจกรรมที่ 2 — การควบคุม LED ให้กะพริบ (Blink LED)

 **วัตถุประสงค์:** ควบคุมการเปิด-ปิด LED ด้วยคำสั่ง digitalWrite ซึ่งเป็นพื้นฐานของการควบคุมเอาต์พุตทั้งหมด

 **อุปกรณ์ที่ใช้:** บอร์ดชุดฝึก (ใช้ LED บนโมดูล ESP32 ที่ขา GPIO2)

```
#define LED_PIN 2          // LED บนโมดูล ESP32

void setup() {
  pinMode(LED_PIN, OUTPUT); // กำหนดขาเป็นเอาต์พุต
}

void loop() {
  digitalWrite(LED_PIN, HIGH); delay(500); // เปิด LED เป็นเวลา 0.5 วินาที
  digitalWrite(LED_PIN, LOW); delay(500);  // ปิด LED เป็นเวลา 0.5 วินาที
}
```

 **ผลการทำงานที่ถูกต้อง:** LED บนโมดูล ESP32 ติดและดับสลับกันทุก 0.5 วินาที

💡 **ข้อแนะนำ:** ทดลองแก้ค่า delay(500) เป็น 100 แล้วอัปโหลดใหม่ LED จะกะพริบเร็วขึ้น 5 เท่า (หากติดตั้งจอ TFT ไว้ให้เปลี่ยน LED_PIN เป็น 5 เนื่องจาก GPIO2 ใช้ร่วมกับจอ)

กิจกรรมที่ 3 — การอ่านค่าสวิตช์กด (Button)

🔗 **วัตถุประสงค์:** อ่านค่าสวิตช์ SW บนบอร์ด พร้อมเรียนรู้หลักการ Active LOW และการป้องกันสัญญาณกระเด็น (Debounce)

🧰 **อุปกรณ์ที่ใช้:** บอร์ดชุดฝึก (สวิตช์ SW บริเวณมุมขวาล่างของบอร์ด = GPIO36)

```
#define SW_PIN 36
bool lastState = HIGH;
int count = 0;

void setup() {
  Serial.begin(115200);
  pinMode(SW_PIN, INPUT);    // GPIO36 ไม่รองรับ INPUT_PULLUP
}

void loop() {
  bool state = digitalRead(SW_PIN);
  if (state == LOW && lastState == HIGH) { // ตรวจจับจังหวะเริ่มกด
    count++;
    Serial.printf("กดครั้งที่ %d\n", count);
    delay(50);                          // ป้องกันสัญญาณกระเด็น
  }
  lastState = state;
}
```

✅ **ผลการทำงานที่ถูกต้อง:** ทุกครั้งที่กดสวิตช์ SW ตัวเลขใน Serial Monitor จะเพิ่มขึ้นครั้งละ 1 อย่างแม่นยำ

📖 **ความรู้เพิ่มเติม:** สวิตช์บนบอร์ดต่อแบบ Active LOW: สถานะปกติอ่านค่าได้ HIGH (มีตัวต้านทาน Pull-up ต่อไว้) เมื่อกดสวิตช์ ขาจะถูกต่อลงกราวด์และอ่านค่าได้ LOW โปรแกรมจึงตรวจสอบเงื่อนไข state == LOW เพื่อแปลว่าสวิตช์ถูกกด

กิจกรรมที่ 4 — การอ่านค่าสัญญาณอนาล็อกจาก VR

🔗 **วัตถุประสงค์:** อ่านค่าอนาล็อกจากตัวต้านทานปรับค่าได้ (VR) และแปลงเป็นค่าแรงดันไฟฟ้าและร้อยละ

🧰 **อุปกรณ์ที่ใช้:** บอร์ดชุดฝึก (VR บริเวณกลางบอร์ด = GPIO34)

```
#define VR_PIN 34
```

```
void setup() { Serial.begin(115200); }

void loop() {
  int raw = analogRead(VR_PIN);           // อ่านค่าได้ 0 - 4095
  float volt = raw * 3.3 / 4095.0;        // แปลงเป็นแรงดันไฟฟ้า (V)
  int percent = map(raw, 0, 4095, 0, 100); // แปลงเป็นร้อยละ 0-100
  Serial.printf("ค่าดิบ: %4d | %.2f V | %3d %%\n", raw, volt, percent);
  delay(200);
}
```

✅ **ผลการทำงานที่ถูกต้อง:** เมื่อหมุน VR ไปทางขวา ค่าที่อ่านได้จะเพิ่มจาก 0 ถึง 4095 (ร้อยละ 0 ถึง 100) และลดลงเมื่อหมุนกลับ

📖 **ความรู้เพิ่มเติม:** ADC (Analog to Digital Converter) ของ ESP32 มีความละเอียด 12 บิต เท่ากับ 4096 ระดับ (0-4095) เทียบกับช่วงแรงดัน 0-3.3V สามารถใช้อ่านค่าจากเซนเซอร์แบบอนาล็อกได้ทุกชนิด

กิจกรรมที่ 5 — การอ่านค่าอินพุตอนาล็อกทั้ง 7 ช่อง

🎯 **วัตถุประสงค์:** อ่านค่าช่องอินพุต IO1–IO7 พร้อมกันทั้งหมด เพื่อเตรียมความพร้อมสำหรับการต่อเซนเซอร์จริง

🛠 **อุปกรณ์ที่ใช้:** บอร์ดชุดฝึก และเซนเซอร์อนาล็อกตามต้องการ (เช่น LDR วัดแสง หรือเซนเซอร์วัดความชื้นดิน) ต่อที่ช่อง IO แบบ S-V-G

1. เปิดไฟล์ 05_AnalogRead_All.ino แล้วกด Upload
2. ทดลองใช้นิ้วแตะขา S ของช่อง IO1 — ค่าของช่อง IO1 จะเปลี่ยนแปลง
3. หากมีเซนเซอร์ให้เสียบที่ช่อง IO ใดก็ได้: ขา S = สัญญาณ, V = ไฟเลี้ยง 5V, G = กราวด์ ตามซิลค์สกรีน

✅ **ผลการทำงานที่ถูกต้อง:** Serial Monitor แสดงค่า IO1–IO7 ทุก 0.5 วินาที ช่องที่ต่อเซนเซอร์จะแสดงค่าที่เปลี่ยนตามสภาพแวดล้อม

💡 **ข้อแนะนำ:** IO6 (GPIO5) อ่านค่าได้เฉพาะแบบดิจิทัล (0/1) และ IO7 (GPIO35) เป็นขารับสัญญาณอย่างเดียว — โปรแกรมตัวอย่างจัดการเงื่อนไขเหล่านี้ให้แล้ว

กิจกรรมที่ 6 — การสร้างสัญญาณ PWM ปรับความสว่าง LED

🎯 **วัตถุประสงค์:** สร้างสัญญาณ PWM เพื่อปรับความสว่างของ LED ให้เพิ่มขึ้นและลดลงอย่างต่อเนื่อง

🛠 **อุปกรณ์ที่ใช้:** บอร์ดชุดฝึก และ LED ต่อที่ขา IO6 (GPIO5) ผ่านตัวต้านทาน 220Ω

```
#define PWM_PIN 5      // IO6 บนบอร์ด

void setup() {
  ledcSetup(0, 5000, 8);    // ช่อง 0, ความถี่ 5kHz, ความละเอียด 8 บิต
  ledcAttachPin(PWM_PIN, 0); // กำหนดขาให้ช่อง PWM
}

void loop() {
  for (int d = 0; d <= 255; d += 5) { ledcWrite(0, d); delay(20); } // สว่างขึ้น
  for (int d = 255; d >= 0; d -= 5) { ledcWrite(0, d); delay(20); } // ลดความสว่าง
}
```

✅ **ผลการทำงานที่ถูกต้อง:** LED ค่อย ๆ สว่างขึ้นจนสูงสุด แล้วค่อย ๆ ลดความสว่างลงจนดับ สลับกันอย่างต่อเนื่อง

📖 **ความรู้เพิ่มเติม:** PWM (Pulse Width Modulation) คือการเปิด-ปิดสัญญาณด้วยความถี่สูงจนสายตามองไม่เห็น สัดส่วนช่วงเวลา "เปิด" ที่มากขึ้นทำให้ LED ดูสว่างขึ้น หลักการเดียวกันนี้ใช้ปรับความเร็วมอเตอร์ในกิจกรรมที่ 8

กิจกรรมที่ 7 — การควบคุมเซอร์โวมอเตอร์ (Servo Sweep)

🎯 **วัตถุประสงค์:** ควบคุมเซอร์โวมอเตอร์สูงสุด 3 ตัว ให้หมุนกวาดไป-กลับในช่วง 0–180 องศา

🧰 **อุปกรณ์ที่ใช้:** บอร์ดชุดฝึก, เซอร์โว SG90/MG90S จำนวน 1–3 ตัว และอะแดปเตอร์ DC 6–12V

1. เสียบเซอร์โวที่ช่อง Servo1 โดยสายสีน้ำตาล/ดำ = G, สีแดง = V, สีส้ม/เหลือง = S
2. ต่ออะแดปเตอร์ DC เข้าบอร์ดแล้วเปิดสวิตช์ ON (เซอร์โวใช้ไฟเลี้ยงจากบอร์ด มิใช่จาก USB)
3. เปิดไฟล์ 07_Servo_Sweep.ino แล้วกด Upload

```
#include <ESP32Servo.h>
Servo servo1;

void setup() {
  servo1.attach(19);    // Servo1 = GPIO19
}

void loop() {
  for (int a = 0; a <= 180; a += 2) { servo1.write(a); delay(15); }
  for (int a = 180; a >= 0; a -= 2) { servo1.write(a); delay(15); }
}
```

✅ **ผลการทำงานที่ถูกต้อง:** แขนเซอร์โวหมุนกวาดจาก 0 ถึง 180 องศา แล้วหมุนกลับอย่างต่อเนื่องและราบรื่น

💡 **ข้อแนะนำ:** สามารถต่อเพิ่มได้ที่ช่อง Servo2 (GPIO32) และ Servo3 (GPIO33) — ทดลองใช้ `servo3.write(180 - a)` เพื่อให้ตัวที่สามหมุนสวนทิศทาง

กิจกรรมที่ 8 — การควบคุมมอเตอร์กระแสตรง (DC Motor)

 **วัตถุประสงค์:** ขับมอเตอร์กระแสตรง 2 ช่องผ่านโมดูล TB67H450: เดินหน้า ถอยหลัง เลี้ยว และปรับความเร็ว

 **อุปกรณ์ที่ใช้:** บอร์ดชุดฝึก, โมดูล TB67H450, มอเตอร์กระแสตรง 1–2 ตัว และอะแดปเตอร์หรือแบตเตอรี่ 6–12V

1. เสียบโมดูล TB67H450 ลงซ็อกเก็ต MOTOR DRIVER ให้ทิศทางตรงตามชิลด์สกรีน
2. ต่อมอเตอร์เข้าเทอร์มินอล M1 (และ M2 กรณีใช้ 2 ตัว)
3. ต่อไฟเลี้ยง DC IN เปิดสวิตช์ ON แล้วเปิดไฟล์ 08_DCMotor_Basic.ino กด Upload

 **ผลการทำงานที่ถูกต้อง:** มอเตอร์หมุนเดินหน้า 2 วินาที → หยุด → ถอยหลัง 2 วินาที → หมุนสวนทางกัน (ลักษณะการเลี้ยว) วนซ้ำต่อเนื่อง

 **ความรู้เพิ่มเติม:** H-Bridge เป็นวงจรสวิตช์ 4 ตัวเรียงกันเป็นรูปตัว H ทำหน้าที่สลับทิศทางกระแสที่ไหลผ่านมอเตอร์ มอเตอร์จึงหมุนได้ทั้งสองทิศทาง และเมื่อจ่ายสัญญาณ PWM แทนสัญญาณคงที่ ก็สามารถปรับความเร็วได้

กิจกรรมที่ 9 — การสแกนหาอุปกรณ์บนบัส I2C (I2C Scanner)

 **วัตถุประสงค์:** ใช้โปรแกรมตรวจสอบหาอุปกรณ์บนบัส I2C ก่อนการใช้งานจอ OLED หรือเซนเซอร์

 **อุปกรณ์ที่ใช้:** บอร์ดชุดฝึก และจอ OLED SSD1306 หรือเซนเซอร์ I2C ต่อที่พอร์ต PH2.0 / ซ็อกเก็ต OLED

1. เสียบจอ OLED ที่ซ็อกเก็ต (GND-VDD-SCK-SDA ตรงตามชิลด์สกรีน)
2. เปิดไฟล์ 09_I2C_Scanner.ino กด Upload แล้วเปิด Serial Monitor

 **ผลการทำงานที่ถูกต้อง:** แสดงข้อความ "Found device at 0x3C" หมายถึงตรวจพบจอ OLED — หากไม่พบ โปรดตรวจสอบการเสียบขาอีกครั้ง

 **ความรู้เพิ่มเติม:** อุปกรณ์ I2C แต่ละชนิดมีหมายเลขประจำตัว (Address) ของตนเอง เช่น OLED = 0x3C, PCA9685 = 0x40, เซนเซอร์วัดแสง BH1750 = 0x23 — บัสเดียวสามารถต่ออุปกรณ์ได้จำนวนมาก หากหมายเลขไม่ซ้ำกัน

กิจกรรมที่ 10 — การเชื่อมต่อเครือข่ายไร้สาย (WiFi Scan & Connect)

🎯 **วัตถุประสงค์:** สแกนหาเครือข่าย WiFi โดยรอบและเชื่อมต่อกับ Router เพื่อเป็นพื้นฐานของงานด้าน IoT

🧰 **อุปกรณ์ที่ใช้:** บอร์ดชุดฝึก และเครือข่าย WiFi ความถี่ 2.4 GHz

1. เปิดไฟล์ 10_WiFi_Scan.ino แก้ไขค่า SSID และ PASSWORD ให้ตรงกับเครือข่ายที่ใช้งาน
2. กด Upload แล้วเปิด Serial Monitor

✅ **ผลการทำงานที่ถูกต้อง:** แสดงรายชื่อเครือข่าย WiFi โดยรอบพร้อมระดับความแรงสัญญาณ (dBm) และเชื่อมต่อสำเร็จพร้อมแสดงหมายเลข IP Address ของบอร์ด

💡 **ข้อแนะนำ:** ESP32 รองรับเฉพาะความถี่ 2.4 GHz — หากไม่พบเครือข่าย โปรดเปิดใช้งานความถี่ 2.4 GHz ที่ Router ก่อน ทั้งนี้ค่า dBm ที่เข้าใกล้ 0 หมายถึงสัญญาณแรง (-40 แรงมาก, -85 อ่อนมาก)

ส่วนที่ 4 — กิจกรรมขั้นสูงและโปรเจกต์ประยุกต์

กิจกรรมที่ 11 — การแสดงค่าอนาล็อกบนจอ TFT

🎯 **วัตถุประสงค์:** ฝึการทำงานของการจอ TFT กับเซนเซอร์: แสดงค่าอนาล็อก 5 ช่องบนจอสีแบบเรียลไทม์

🧰 **อุปกรณ์ที่ใช้:** บอร์ดชุดฝึก และจอ ST7789 (พร้อมเซนเซอร์ตามต้องการ)

เปิดไฟล์ Arduino_Example_Advance/LCD_AnalogInput แล้วกด Upload — จอจะแสดงหัวข้อ Analog Monitor และค่า AIN1–AIN5 ซึ่งอัปเดตทุก 0.5 วินาที ทดลองหมุน VR หรือแตะขาเซนเซอร์ เพื่อสังเกตการเปลี่ยนแปลงของค่า

✅ **ผลการทำงานที่ถูกต้อง:** หน้าจอแสดงค่าเซนเซอร์ทั้ง 5 ช่องแบบเรียลไทม์ โดยไม่ต้องใช้ Serial Monitor บนคอมพิวเตอร์

กิจกรรมที่ 12 — การใช้จอ OLED

🎯 **วัตถุประสงค์:** ศึกษาความสามารถของจอ OLED: การวาดเส้น รูปทรงเรขาคณิต ข้อความ และภาพเคลื่อนไหว

🧰 **อุปกรณ์ที่ใช้:** บอร์ดชุดฝึก และจอ OLED SSD1306

เปิดไฟล์ Arduino_Example_Advance/OLED_I2C แล้วกด Upload — จอจะแสดงตัวอย่างที่ละรูปแบบ ได้แก่ เส้นตรง สีเหลี่ยม วงกลม สามเหลี่ยม ตัวอักษร ข้อความเลื่อน และภาพเคลื่อนไหว

✅ **ผลการทำงานที่ถูกต้อง:** จอ OLED แสดงกราฟิกครบทุกรูปแบบโดยอัตโนมัติ — โปรแกรมนี้เป็นแหล่งอ้างอิงวิธีการวาดกราฟิกแต่ละชนิด

กิจกรรมที่ 13 — การควบคุมเซอร์โวด้วย VR

🎯 **วัตถุประสงค์:** ใช้ VR ควบคุมมุมของเซอร์โว 2 ตัวแบบเรียลไทม์ และใช้สวิตช์ SW สลับสถานะ LED

🧰 **อุปกรณ์ที่ใช้:** บอร์ดชุดฝึก, เซอร์โว 2 ตัว และอะแดปเตอร์ DC

เปิดไฟล์ Arduino_Example_Advance/ServoMotor แล้วกด Upload — เมื่อหมุน VR อย่างช้า ๆ เซอร์โวตัวแรกจะหมุนตาม ส่วนตัวที่สองหมุนสวนทิศทาง (มุม 180-a) และกดสวิตช์ SW เพื่อเปิด-ปิด LED

✅ **ผลการทำงานที่ถูกต้อง:** เซอร์โวเคลื่อนที่ตามการหมุน VR ทันที ซึ่งเป็นหลักการเดียวกับอุปกรณ์ควบคุมแขนกลในงานจริง

กิจกรรมที่ 14 — โปรแกรมทดสอบการทำงาน (Factory Test)

 **วัตถุประสงค์:** ใช้โปรแกรมทดสอบจากโรงงานเพื่อตรวจสอบการทำงานของทุกฟังก์ชันบนบอร์ดภายในโปรแกรมเดียว

 **อุปกรณ์ที่ใช้:** บอร์ดชุดฝึกครบชุด: จอ TFT, TB67H450, มอเตอร์, เซอร์โว และอะแดปเตอร์ DC

เปิดไฟล์ Arduino_Example_Advance/SKU_1015_ESP32_FACTORY_TEST แล้วกด Upload — โปรแกรมแสดงผลบนจอ TFT แบ่งเป็น 3 หน้า กดสวิตช์ SW เพื่อเปลี่ยนหน้า ได้แก่ หน้า 1 ค่า เซนเซอร์ 7 ช่องพร้อมกราฟแท่ง หน้า 2 ค่า VR และเซอร์โว 3 ตัวที่เคลื่อนที่ตาม VR และหน้า 3 การทดสอบมอเตอร์เดินหน้า-หยุด-ถอยหลังโดยอัตโนมัติ

 **ผลการทำงานที่ถูกต้อง:** ทุกหน้าทำงานครบถ้วน แสดงว่าบอร์ดอยู่ในสภาพสมบูรณ์ — สามารถใช้โปรแกรมนี้อ้างอิงตรวจสอบบอร์ดหลังแกะกล่องหรือก่อนการแข่งขันได้

โปรเจกต์ประยุกต์ — รถหุ่นยนต์ขับเคลื่อน 2 ล้อ

 **วัตถุประสงค์:** ประยุกต์ความรู้จากกิจกรรมที่ 1-14 เพื่อสร้างรถหุ่นยนต์ที่ขับเคลื่อนได้จริง

 **อุปกรณ์ที่ใช้:** ชุดฝึกครบชุด, โครงรถ 2 ล้อ, มอเตอร์พร้อมเกียร์ 2 ตัว และแบตเตอรี่ 7.4V (Li-ion 2S) หรือถ่าน AA 6 ก้อน

1. ประกอบโครงรถ: มอเตอร์ล้อซ้ายต่อเข้า M1, ล้อขวาต่อเข้า M2 แล้วยึดบอร์ดบนโครงด้วยรูดึงทั้ง 4 มุม
2. ต่อแบตเตอรี่เข้าช่อง DC IN (โปรดตรวจสอบขั้ว + / - ให้ถูกต้อง)
3. อัปโหลดโปรแกรมด้านล่าง วางรถบนพื้น แล้วเปิดสวิตช์ ON

```
// รถหุ่นยนต์ 2 ล้อ: ล้อซ้าย = M1, ล้อขวา = M2
#define M1_IN1 27
#define M1_IN2 26
#define M2_IN1 16
#define M2_IN2 17

void motorControl(int motor, int speed) {          // speed -255..255
    speed = constrain(speed, -255, 255);
    int chA = (motor == 1) ? 0 : 2, chB = (motor == 1) ? 1 : 3;
    if (speed >= 0) { ledcWrite(chA, speed); ledcWrite(chB, 0); }
    else          { ledcWrite(chA, 0);    ledcWrite(chB, -speed); }
}

void drive(int left, int right) { motorControl(1, left); motorControl(2, right); }

void setup() {
    ledcSetup(0, 1000, 8); ledcAttachPin(M1_IN1, 0);
```



```

    ledcSetup(1, 1000, 8); ledcAttachPin(M1_IN2, 1);
    ledcSetup(2, 1000, 8); ledcAttachPin(M2_IN1, 2);
    ledcSetup(3, 1000, 8); ledcAttachPin(M2_IN2, 3);
}

void loop() {
    drive(200, 200);    delay(1500);    // เดินหน้า
    drive(-200, -200);  delay(1000);    // ถอยหลัง
    drive(200, -200);   delay(700);     // หมุนตัวไปทางขวา
    drive(0, 0);        delay(500);     // หยุด
}

```

✅ **ผลการทำงานที่ถูกต้อง:** รถเคลื่อนที่เดินหน้า ถอยหลัง หมุนตัว และหยุด วนซ้ำตามลำดับ — การสร้างหุ่นยนต์เคลื่อนที่พื้นฐานเสร็จสมบูรณ์

💡 **ข้อแนะนำ:** แนวทางต่อยอด: ติดตั้งเซนเซอร์วัดระยะที่ช่อง IO1 และเขียนเงื่อนไขให้เลี้ยวหลบเมื่อระยะน้อยกว่า 20 เซนติเมตร จะได้หุ่นยนต์หลบสิ่งกีดขวาง / ติดตั้งเซนเซอร์ตรวจจับเส้น 2 ตัว จะได้หุ่นยนต์เดินตามเส้น / เพิ่มจอ TFT เพื่อแสดงสถานะการทำงาน

ส่วนที่ 5 — แหล่งข้อมูลอ้างอิงและการแก้ไขปัญหา

5.1 ลิงก์สินค้าทั้ง 4 รายการ

สินค้า	SKU / ราคา	ลิงก์
บอร์ดขยาย ESP32 38PIN Breakout Board	SKU1015 / 590.-	massmore.shop/products/1cbd7c12-166a-4cbb-bbc6-3d7fdce0775c
บอร์ดพัฒนา ESP32-WROOM-32E 38PIN CH340	SKU1016 / 350.-	massmore.shop/products/819d35cb-3693-4d0f-8b10-4ff37821161f
โมดูลขับเคลื่อนมอเตอร์ 2CH TB67H450	SKU1002 / 159.-	massmore.shop/products/6429a185-604c-4f1e-9dfd-59745e82258b
จอ TFT LCD IPS 240x240 ST7789	SKU0014 / 160.-	massmore.shop/products/0058df49-88b0-4422-b0b2-0ddc18beafa2

5.2 GitHub และไลบรารี

รายการ	URL
โปรเจกต์นี้ (โปรแกรม คู่มือ และไลบรารี)	github.com/Massmore/SKU-1015_ESP32BreakoutBoard
ESP32 Arduino Core	github.com/espressif/arduino-esp32
TFT_eSPI (Bodmer)	github.com/Bodmer/TFT_eSPI
ESP32Servo	github.com/madhephaestus/ESP32Servo
Adafruit SSD1306 / GFX	github.com/adafruit/Adafruit_SSD1306

5.3 เอกสาร Datasheet

เอกสาร	แหล่งที่มา
ESP32-WROOM-32E/32UE Datasheet	โฟลเดอร์ Document/Datasheet หรือ espressif.com
ESP32 Technical Reference Manual	espressif.com/sites/default/files/documentation/esp32_technical_reference_manual_en.pdf
CH340 Datasheet	Document/Datasheet/CH340DS1.PDF
TB67H450 Datasheet (TOSHIBA)	toshiba.semicon-storage.com (หรือ Document/Datasheet)
ST7789 Datasheet	newhavendisplay.com/appnotes/datasheets/LCDs/ST7789V.pdf

5.4 การแก้ไขปัญหาที่พบบ่อย (Troubleshooting)

อาการ	สาเหตุที่เป็นไปได้	แนวทางแก้ไข
ไม่พบพอร์ต COM	ยังไม่ได้ติดตั้ง Driver CH340 / สาย USB ใช้ชาร์จได้อย่างเดียว	ติดตั้ง Driver (ส่วนที่ 2 ขั้นตอนที่ 2) และเปลี่ยนสาย USB
อัปโหลดค้างที่ Connecting...	บอร์ดไม่เข้าสู่โหมด Download	กดปุ่ม BOOT ค้างระหว่างอัปโหลด / ลดค่า Upload Speed เป็น 115200
Serial Monitor ไม่แสดงข้อมูล	ค่า Baud rate ไม่ตรงกัน	ตั้งค่า 115200 ให้ตรงกับโปรแกรม
จอ TFT ไม่แสดงผล	การตั้งค่า User_Setup.h ไม่ถูกต้อง	ใช้ TFT_eSPI จากโฟลเดอร์ของโปรเจกต์
จอ TFT สีสันเพี้ยน	ลำดับสีหรือ Inversion ไม่ถูกต้อง	ต้องกำหนด TFT_RGB_ORDER TFT_BGR และ TFT_INVERSION_ON
จอ OLED ไม่แสดงผล	Address ไม่ถูกต้อง / การเสียบขาไม่แน่น	รันกิจกรรมที่ 9 ต้องตรวจพบ 0x3C
เซอร์โวลั่นหรือไม่ทำงาน	ไฟเลี้ยงไม่เพียงพอ	ต่อ DC IN 6-12V ทุกครั้ง ไม่ใช่ไฟจาก USB เพียงอย่างเดียว
มอเตอร์ไม่หมุน	ไม่ได้ติดตั้งโมดูล Driver / ไม่ได้เปิดสวิตช์	ตรวจสอบโมดูล ต่อไฟเลี้ยง และเปิดสวิตช์ ON
สวิตช์ SW ไม่ทำงาน	ใช้ INPUT_PULLUP กับ GPIO36	GPIO36 ต้องใช้ pinMode(36, INPUT)
คำสั่ง ledcSetup คอมไพล์ไม่ผ่าน	ติดตั้ง core เวอร์ชัน 3.x	เปลี่ยนกลับมาใช้ esp32 core 2.0.17

5.5 ข้อแนะนำเพิ่มเติม (Pro Tips)

- การจ่าย PWM ให้มอเตอร์ที่ความถี่สูงกว่า 20 kHz จะช่วยยืดอายุการใช้งานของมอเตอร์ ช่วยลดเสียงรบกวนจากมอเตอร์
- เพื่อให้ค่า ADC มีความนิ่งมากขึ้น ควรอ่านค่าซ้ำประมาณ 10 ครั้งแล้วคำนวณค่าเฉลี่ย เพื่อลดผลของสัญญาณรบกวน
- กรณีโปรแกรมหมีขนาดใหญ่เกินพื้นที่ Flash ให้เปลี่ยน Partition Scheme เป็น Huge APP (3MB No OTA)
- งานที่ใช้แบตเตอรี่ ควรใช้คำสั่ง esp_deep_sleep_start() เพื่อลดการใช้พลังงานเหลือระดับไมโครแอมแปร์
- การต่อยอดด้าน IoT: สามารถส่งค่าเซนเซอร์ไปยัง Blynk / Firebase / MQTT ได้ โดยอาศัยพื้นฐานจากกิจกรรมที่ 10

